Detailed walk-through with concrete example

We will use these settings so the math is easy to follow:

- `context_size = 2`
- Context tokens: `"roof age"`
- Target (what we want to predict): `"greater"`
- `embedding_dim = 5` (very small for learning)
- `vocab_size = 6` (small vocabulary for illustration)
- Batch size = 1

Assumed vocabulary & IDs

Token	ID
pad	0
roof	1
age	2
greater	3
than	4
10	5

So the input becomes:

```python
x = torch.tensor([[1, 2]])   # shape: [1, 2]   (batch=1, context_size=2)
```

Step-by-step Forward Pass with Pictures & Math

1. Input

```text
```

```
x
┌──────────────┐
│  1     2     │      shape: [1, 2]
└──────────────┘
  roof   age
```

---

2. Embedding Lookup

```
Python
```

```
embeddings = self.embedding(x)
```

The embedding layer is a big lookup table of shape `[vocab_size, 5]`.

Let's pretend the embedding table looks like this (I made up small numbers):

```
text
```

```
Embedding Table (vocab_size × 5)
──────────────────────────────────────
ID 0 (pad)     :  0.0   0.0   0.0   0.0   0.0
ID 1 (roof)    :  0.2  -0.1   0.4   0.3  -0.2
ID 2 (age)     : -0.3   0.5   0.1  -0.4   0.2
ID 3 (greater) :  0.1   0.2  -0.3   0.5   0.0
ID 4 (than)    : ...
ID 5 (10)      : ...
```

After lookup:

```
text
```

```
embeddings = self.embedding(x)          # shape: [1, 2, 5]

┌──────────────────────────────────────────────┐
│  roof embedding:   [ 0.2, -0.1,  0.4,  0.3, -0.2 ]  │
│  age  embedding:   [-0.3,  0.5,  0.1, -0.4,  0.2 ]  │
└──────────────────────────────────────────────┘
```

Picture:

```text
[batch, context, dim]
     ↓
┌──────────────────────────────────┐
│  [0.2, -0.1, 0.4, 0.3, -0.2] │  ← roof
│  [-0.3, 0.5, 0.1, -0.4, 0.2] │  ← age
└──────────────────────────────────┘
```

---

3. Average the context embeddings

```python
embeddings = embeddings.mean(dim=1)
```

We average along the context dimension (dim=1).

Math:

$$\text{mean} = \frac{1}{2} \times \left(\begin{bmatrix} 0.2 \\ -0.1 \\ 0.4 \\ 0.3 \\ -0.2 \end{bmatrix} + \begin{bmatrix} -0.3 \\ 0.5 \\ 0.1 \\ -0.4 \\ 0.2 \end{bmatrix} \right)$$

$$= \begin{bmatrix} \frac{0.2+(-0.3)}{2} \\ \frac{-0.1+0.5}{2} \\ \frac{0.4+0.1}{2} \\ \frac{0.3+(-0.4)}{2} \\ \frac{-0.2+0.2}{2} \end{bmatrix} = \begin{bmatrix} -0.05 \\ 0.20 \\ 0.25 \\ -0.05 \\ 0.00 \end{bmatrix}$$

Result shape: `[1, 5]`

```text
```

```
Averaged vector (context summary):
[ -0.05,  0.20,  0.25, -0.05,  0.00 ]
```

Picture:

```
text
```

```
Before mean:          After mean:
[2 vectors]    →      [1 vector of size 5]
```

This single vector is now a summary of the entire context `"roof age"`.

---

4. Linear Layer (the prediction head)

```
Python
```

```
output = self.linear(embeddings)
```

The linear layer is:

$$\text{output} = \text{embeddings} \times W^T + b$$

Where:

- `W` has shape `[vocab_size, embedding_dim] = [6, 5]`
- `b` has shape `[6]`

Let's invent simple weights for illustration:

```
text
```

```
Weight matrix W (6 × 5):
Row 0 (for token 0):  [ 0.1,  0.2, -0.1,  0.0,  0.3]
Row 1 (for token 1):  [-0.2,  0.1,  0.4, -0.3,  0.0]
Row 2 (for token 2):  [ 0.3, -0.1,  0.2,  0.1, -0.2]
Row 3 (for token 3):  [ 0.4,  0.3, -0.2,  0.5,  0.1]   ← this is for
"greater"
Row 4 (for token 4):  [-0.1,  0.0,  0.3, -0.4,  0.2]
Row 5 (for token 5):  [ 0.2, -0.3,  0.1,  0.0, -0.1]
```

Bias `b = [0.0, 0.1, -0.1, 0.2, 0.0, -0.2]`

Now the matrix multiplication:

$$\text{logits} = \begin{bmatrix} -0.05 & 0.20 & 0.25 & -0.05 & 0.00 \end{bmatrix} \times W^T + b$$

After calculation you get 6 numbers (one score per vocabulary word):

```
text
```

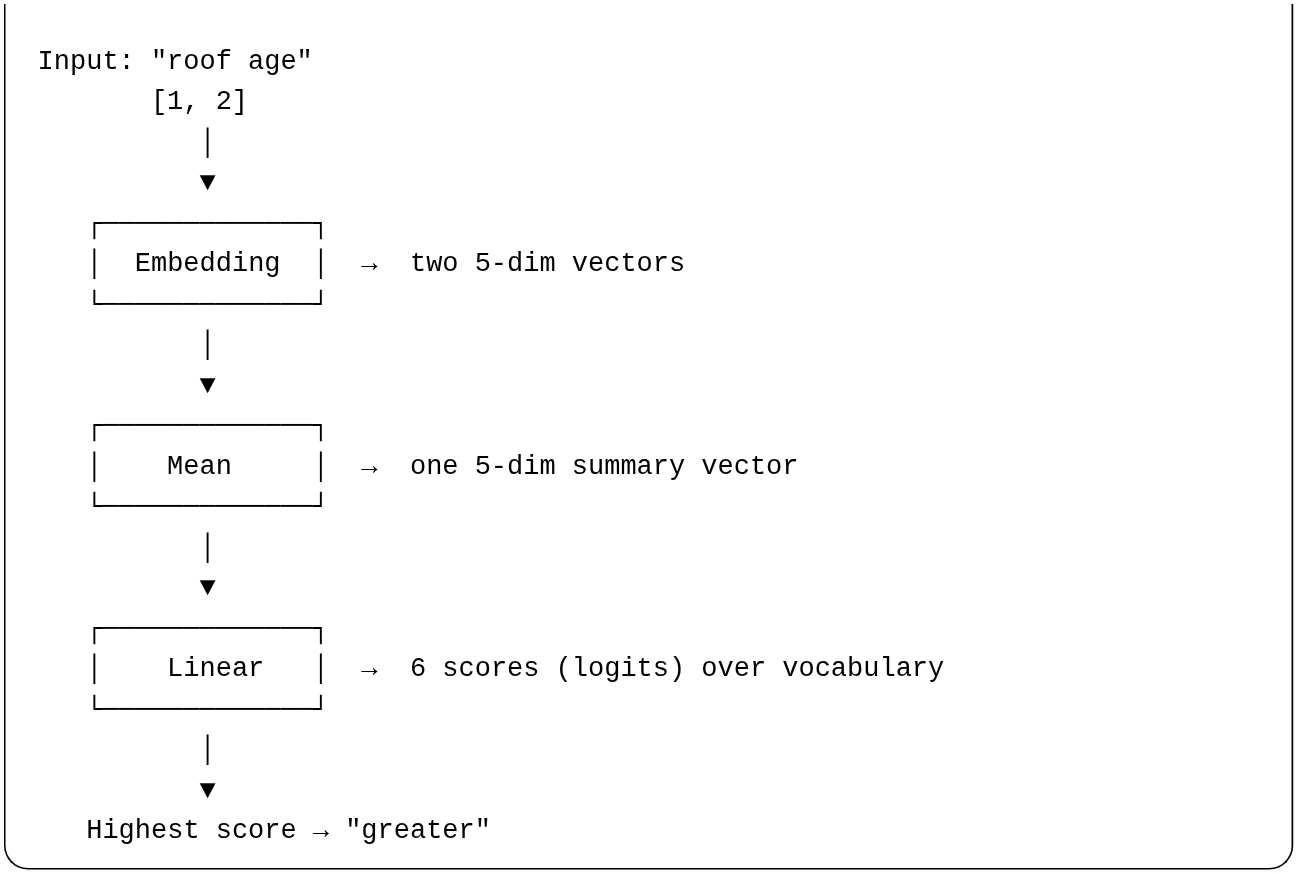
```
output (logits) shape: [1, 6]

[  0.12,   -0.05,   0.08,   0.31,  -0.07,   0.04 ]
   ↑        ↑        ↑       ↑       ↑       ↑
  pad     roof     age   greater   than     10
```

The highest score is **0.31** for token ID 3 → `"greater"` .

This is exactly what we hoped the model would predict as the next word!

Full Flow Summary (Pictorial)

```
text
```



Key Mathematical Ideas

1. **Embedding** = just a lookup table (no calculation, only indexing).
2. **Mean** = simple average → compresses variable-length context into fixed size.
3. **Linear** = matrix multiplication that maps the summary vector into vocabulary space so we can score every possible next word.

Would you like me to also show the same example with the actual PyTorch code + how the loss would be calculated against the target `"greater"`?